The default and most common Index type used is a B-tree Index.[1] A B-tree index is a secondary data structure represented as a tree optimized for retrieval speed.

B-tree indexes are used for equality comparisons. Each node points to a row. A query using the Index may be searching for rows with matching column values to filter down a result set.

When an Index was scanned and used to identify the row, as you saw earlier the query execution plan will name the index and show that an Index Scan was performed. The index contains a reference to the row identifier. Using the row identifier called the ctid, the table or heap is then scanned to retrieve the row with the columns specified in the query.

With that refresher in place, it's time to work with a less common Index that transforms data before being stored.

Generating Data for Experiments

One of the best ways to learn about the behavior of PostgreSQL is to create experiments by generating test data and performing queries and running commands on large sets of data. To work with Indexes locally on large sized data, open up psql from your Terminal and run these SQL statements:

sql/insert_users_generate_series.sql

```
CREATE SCHEMA IF NOT EXISTS temp;

DROP TABLE IF EXISTS temp.users;

CREATE TABLE IF NOT EXISTS temp.users AS
SELECT
  'fname' || seq AS first_name,
  'lname' || seq AS last_name,
  'user_' || seq || '@' || (
    CASE (RANDOM() * 2)::INT
      WHEN 0 THEN 'gmail'
      WHEN 1 THEN 'hotmail'
      WHEN 2 THEN 'yahoo'
    END
  ) || '.com' AS email,
  CASE (seq % 2)
    WHEN 0 THEN 'Driver'
    ELSE 'Rider'
  END AS type,
  NOW() AS created_at,
  NOW() AS updated_at
FROM GENERATE_SERIES(1, 10000000) seq;
```

1. https://www.postgresql.org/docs/current/btree-implementation.html